Using Query:

```
postgres=# SELECT * FROM students WHERE grade IS NULL;
 id | name | age | grade
----+------+-----+-------
(0 rows)


postgres=# SELECT * FROM students WHERE grade IS NOT NULL;
 id |  name   | age | grade
----+---------+-----+-------
  1 | Alice   |  20 | A
  2 | Bob     |  22 | B
  3 | Charlie |  19 | C
  4 | David   |  21 | A
  5 | Eve     |  23 | B
(5 rows)
```

```
postgres=# SELECT * FROM students WHERE name LIKE 'A%';
 id | name  | age | grade
----+-------+-----+-------
  1 | Alice |  20 | A
(1 row)


postgres=# SELECT * FROM students WHERE name LIKE '%e';
 id |  name   | age | grade
----+---------+-----+-------
  1 | Alice   |  20 | A
  3 | Charlie |  19 | C
  5 | Eve     |  23 | B
(3 rows)


postgres=# SELECT * FROM students WHERE name LIKE '_o%';
 id | name | age | grade
----+------+-----+-------
  2 | Bob  |  22 | B
(1 row)
```

```
postgres=# -- BETWEEN (inclusive)
postgres=# SELECT * FROM students WHERE age BETWEEN 20 AND 22;
 id | name  | age | grade
----+-------+-----+-------
  1 | Alice |  20 | A
  2 | Bob   |  22 | B
  4 | David |  21 | A
(3 rows)


postgres=#
postgres=# -- NOT BETWEEN
postgres=# SELECT * FROM students WHERE age NOT BETWEEN 19 AND 21;
 id | name | age | grade
----+------+-----+-------
  2 | Bob  |  22 | B
  5 | Eve  |  23 | B
(2 rows)


postgres=# SELECT * FROM students WHERE grade IN ('A', 'C');
 id |  name   | age | grade
----+---------+-----+-------
  1 | Alice   |  20 | A
  3 | Charlie |  19 | C
  4 | David   |  21 | A
(3 rows)


postgres=# SELECT * FROM students WHERE grade NOT IN ('B');
 id |  name   | age | grade
----+---------+-----+-------
  1 | Alice   |  20 | A
  3 | Charlie |  19 | C
  4 | David   |  21 | A
(3 rows)
```

```
postgres=#
postgres=# -- NOT (negation)
postgres=# SELECT * FROM students WHERE NOT grade = 'C';
 id | name  | age | grade
----+-------+-----+-------
  1 | Alice |  20 | A
  2 | Bob   |  22 | B
  4 | David |  21 | A
  5 | Eve   |  23 | B
(4 rows)


postgres=# -- BETWEEN (inclusive)
postgres=# SELECT * FROM students WHERE age BETWEEN 20 AND 22;
 id | name  | age | grade
----+-------+-----+-------
  1 | Alice |  20 | A
  2 | Bob   |  22 | B
  4 | David |  21 | A
(3 rows)


postgres=#
postgres=# -- NOT BETWEEN
postgres=# SELECT * FROM students WHERE age NOT BETWEEN 19 AND 21;
 id | name | age | grade
----+------+-----+-------
  2 | Bob  |  22 | B
  5 | Eve  |  23 | B
(2 rows)
```

```
postgres=# -- AND (both conditions true)
postgres=# SELECT * FROM students WHERE age > 20 AND grade = 'B';
 id | name | age | grade
----+------+-----+-------
  2 | Bob  |  22 | B
  5 | Eve  |  23 | B
(2 rows)


postgres=#
postgres=# -- OR (any condition true)
postgres=# SELECT * FROM students WHERE age < 20 OR grade = 'A';
 id |  name   | age | grade
----+---------+-----+-------
  1 | Alice   |  20 | A
  3 | Charlie |  19 | C
  4 | David   |  21 | A
(3 rows)


postgres=#
postgres=# -- NOT (negation)
postgres=# SELECT * FROM students WHERE NOT grade = 'C';
 id | name  | age | grade
----+-------+-----+-------
  1 | Alice |  20 | A
  2 | Bob   |  22 | B
  4 | David |  21 | A
  5 | Eve   |  23 | B
(4 rows)
```

```
postgres=# SELECT * FROM students;
 id |  name   | age | grade
----+---------+-----+-------
  1 | Alice   |  20 | A
  2 | Bob     |  22 | B
  3 | Charlie |  19 | C
  4 | David   |  21 | A
  5 | Eve     |  23 | B
(5 rows)


postgres=# SELECT * FROM students WHERE age = 20;
 id | name  | age | grade
----+-------+-----+-------
  1 | Alice |  20 | A
(1 row)


postgres=# SELECT * FROM students WHERE grade <> 'A';
 id |  name   | age | grade
----+---------+-----+-------
  2 | Bob     |  22 | B
  3 | Charlie |  19 | C
  5 | Eve     |  23 | B
(3 rows)


postgres=# SELECT * FROM students WHERE age > 21;
 id | name | age | grade
----+------+-----+-------
  2 | Bob  |  22 | B
  5 | Eve  |  23 | B
(2 rows)


postgres=# SELECT * FROM students WHERE age <= 20;
 id |  name   | age | grade
----+---------+-----+-------
  1 | Alice   |  20 | A
  3 | Charlie |  19 | C
(2 rows)
```

```
postgres=# CREATE TABLE students (
postgres(#     id SERIAL PRIMARY KEY,
postgres(#     name VARCHAR(50),
postgres(#     age INT,
postgres(#     grade CHAR(1)
postgres(# );
ERROR:  relation "students" already exists
postgres=# DROP TABLE students;
DROP TABLE
postgres=#
postgres=# CREATE TABLE students (
postgres(#     id SERIAL PRIMARY KEY,
postgres(#     name VARCHAR(50),
postgres(#     age INT,
postgres(#     grade CHAR(1)
postgres(# );
CREATE TABLE
postgres=# INSERT INTO students (name, age, grade) VALUES
postgres-# ('Alice', 20, 'A'),
postgres-# ('Bob', 22, 'B'),
postgres-# ('Charlie', 19, 'C'),
postgres-# ('David', 21, 'A'),
postgres-# ('Eve', 23, 'B');
INSERT 0 5
```

```
postgres=# INSERT INTO students (name, age) VALUES
postgres-# ('Alice', 20),
postgres-# ('Bob', 22),
postgres-# ('Charlie', 19);
INSERT 0 3
postgres=# SELECT * FROM students;
 id |  name   | age
----+---------+-----
  1 | Alice   |  20
  2 | Bob     |  22
  3 | Charlie |  19
(3 rows)
```

Python Code:

```
import psycopg2

conn = psycopg2.connect(
    dbname="postgres",
    user="postgres",
    password="ansh123",
    host="localhost",
    port="5432"
)
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    grade CHAR(1)
)
""")
cur.execute("TRUNCATE TABLE students;")
cur.execute("""
INSERT INTO students (name, age, grade) VALUES
('Alice', 20, 'A'),
('Bob', 22, 'B'),
('Charlie', 19, 'C'),
```

```
('David', 21, 'A'),
('Eve', 23, 'B')
""")
conn.commit()

print("\n1. Students age > 20:")
cur.execute("SELECT * FROM students WHERE age > 20;")
print(cur.fetchall())

print("\n2. Students with grade = 'A':")
cur.execute("SELECT * FROM students WHERE grade = 'A';")
print(cur.fetchall())

print("\n3. Students with age BETWEEN 20 AND 22:")
cur.execute("SELECT * FROM students WHERE age BETWEEN 20 AND 22;")
print(cur.fetchall())

print("\n4. Students with name LIKE 'A%':")
cur.execute("SELECT * FROM students WHERE name LIKE 'A%';")
print(cur.fetchall())

print("\n5. Students with grade IN ('A','C'):")
cur.execute("SELECT * FROM students WHERE grade IN ('A','C');")
print(cur.fetchall())
cur.close()
conn.close()
```

Output:

```
PS E:\Python Programs\Python Course\Assignment-7> pip install psycopg2
>>
Collecting psycopg2
  Downloading psycopg2-2.9.10-cp313-cp313-win_amd64.whl.metadata (4.8 kB)
Downloading psycopg2-2.9.10-cp313-cp313-win_amd64.whl (2.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.6/2.6 MB 5.6 MB/s  0:00:00
Installing collected packages: psycopg2
Successfully installed psycopg2-2.9.10
```

```
PS E:\Python Programs\Python Course\Assignment-7> python -u "e:\Python Programs\Python Course\Assignment-7\database.py"

1. Students age > 20:
[(7, 'Bob', 22, 'B'), (9, 'David', 21, 'A'), (10, 'Eve', 23, 'B')]

2. Students with grade = 'A':
[(6, 'Alice', 20, 'A'), (9, 'David', 21, 'A')]

3. Students with age BETWEEN 20 AND 22:
[(6, 'Alice', 20, 'A'), (7, 'Bob', 22, 'B'), (9, 'David', 21, 'A')]

4. Students with name LIKE 'A%':
[(6, 'Alice', 20, 'A')]

5. Students with grade IN ('A','C'):
[(6, 'Alice', 20, 'A'), (8, 'Charlie', 19, 'C'), (9, 'David', 21, 'A')]
```